MIT 8103 Advanced Database Systems: Portfolio Documentation

Sunkanmi Samson Otitolaye | Master of Information Technology, MIVA Open University | First Semester CA 2026/2027 | **GitHub (private):** https://github.com/leosuky/advanced-database-portfolio

1. Selected case study

The Data XI is a football analytics platform I am building. This portfolio uses a **simulated** copy of its database (seeded generator, invented clubs and players, no real provider data). The warehouse stores competitions, seasons, teams, players, matches, player and team statistics, shots and about 1.14 million on-ball events (3 seasons, 20 teams, 1,140 matches). Its users are the ingestion pipeline (writes), the public API (read-only, repeated queries) and analysts (heavy reads). Key operations: load matches, correct a score and keep the league table right, leaderboards, pass networks, cached match pages, surviving a server failure and spreading the biggest table over servers. The same case study is used in all five activities.

2. Portfolio 1: database design and modelling

A 17-table PostgreSQL 16 schema in 3NF/BCNF with primary, foreign and unique keys and CHECK constraints that state business rules (a finished match must have a score; standings points must equal 3 x wins + draws). A composite foreign key (match, team) guarantees a player can only be attached to a team that played that match. Provider reconciliation (four providers, one canonical team id) is modelled with a mapping table. Denormalisation is deliberate and documented: the score on match, team_id on shot/event, and a derived standings table, each protected by a cross-table integrity check. Evidence: nine constraint tests all rejected as expected; four integrity checks return zero rows.

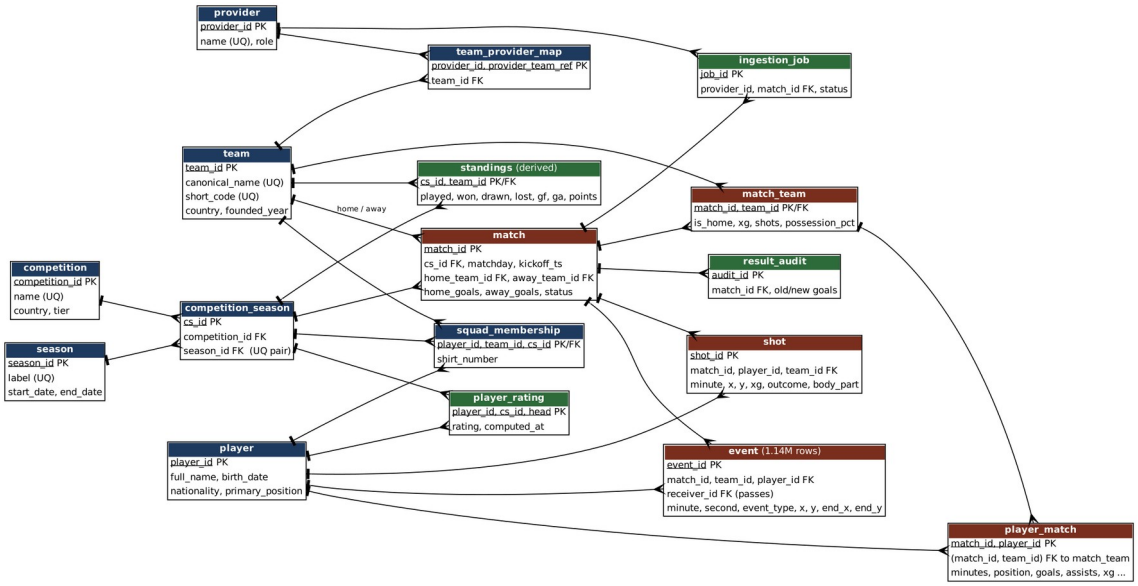


Figure 1. Entity-relationship diagram (blue: reference, brown: facts, green: derived and operational).

3. Portfolio 2: query processing and optimisation

Nine queries were timed (median of 14 warm runs) with EXPLAIN (ANALYZE, BUFFERS), before and after adding indexes. Postgres does not index foreign keys automatically, so the pass-network query scanned all 1,140,000 events (12,960 buffers) to return 476 rows; one index on event(match_id) reduced it to 16 buffers. A covering index gave an index-only scan (Q5), and a partial index covering only the 2,777 goals is 48 kB against 240 kB for a full index (Q8). The largest gain on Q7 was not an index: rewriting a correlated subquery as a window function cut 109,508 buffer hits to 311 and 478 ms to 4.6 ms. Indexes on the small tables (Q1 to Q4) gave no measurable benefit, so I judge three of them unnecessary at this size. A materialised view answered the leaderboard in 0.3 ms instead of 3.8 ms, at the cost of staleness. The planner's cheapest estimate (Hash Join) was not the fastest measured join, a reminder to measure rather than trust costs.

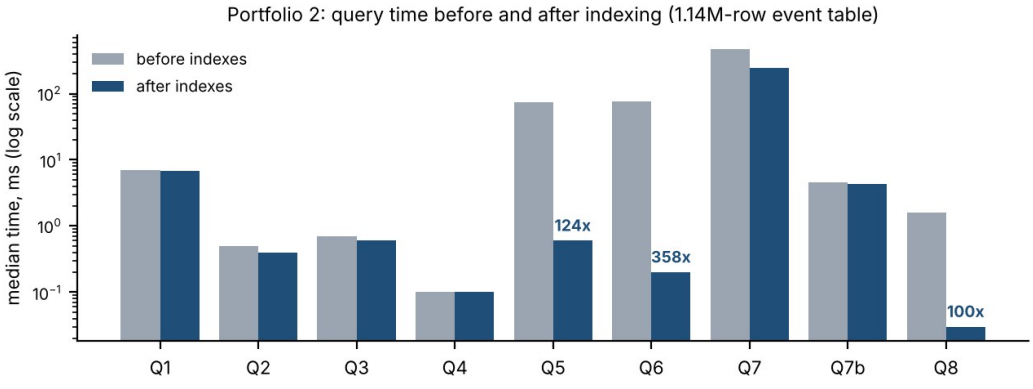


Figure 2. Before and after timings (log scale). Labels show the measured speed-up.

4. Portfolio 3: transactions and concurrency

A score correction changes the match, both teams' standings and an audit row in one ACID transaction (correct_result). Six scenarios ran with real concurrent sessions:

Scenario	Result
Commit / rollback	Other sessions saw the old score until commit; a failed second step rolled back the first; a savepoint undid only the bad job (2 of 3 stored).
Isolation levels	READ COMMITTED showed a non-repeatable read (37 to 35 points) and a phantom row; REPEATABLE READ gave identical reads but refused a conflicting write (serialisation failure, retry needed); no dirty reads exist in PostgreSQL.
Lost update	Two workers each added a win by read-modify-write: 9 recorded instead of 10, silently. SELECT … FOR UPDATE and a single atomic UPDATE both gave 10.
Deadlock	Opposite lock order: PostgreSQL aborted one transaction. A fixed lock order (lowest team id first) let both commit.
Write skew	A rule of at most 2 running jobs was broken (3 running) under REPEATABLE READ and held under SERIALIZABLE, which aborted one transaction.

5. Portfolio 4: NoSQL and advanced data models

Key-value (Redis): a sorted set serves the scorers leaderboard in 0.52 ms against 2.31 ms from PostgreSQL; a hash with pub/sub carries live match state (HINCRBY is atomic); a TTL string implements cache-aside (99% hit rate); INCR/EXPIRE rate-limits an API key. The cost was shown too: after a score change the cache stayed stale until invalidated (eventual consistency, BASE). **Document (MongoDB):** a match page is one 5.1 KB document with embedded line-ups and shots, and raw payloads from four providers with four different shapes sit in one schema-flexible collection, with a JSON-schema validator on the page documents. The top-scorer pipeline agrees with PostgreSQL. On a real MongoDB 7 server the validator rejected an invalid document (code 121) and explain() showed an index scan examining 19 documents instead of 1,140. Running on the real server also exposed three bugs that an in-memory emulator had hidden. A graph model for pass networks is discussed but not implemented.

6. Portfolio 5: distributed and cloud database exercise

Replication: a real primary and streaming replica showed read-only standby behaviour, lag after a 400,000-row burst (replica showed 0 rows at first), and commit cost of 0.30 ms asynchronous against 0.58 ms synchronous. With the replica down, a synchronous commit waited indefinitely (consistency over availability). After a primary crash under asynchronous replication, an acknowledged commit was lost (RPO > 0) and the old primary came back as a second primary (split brain); under synchronous replication nothing was lost. **Sharding:** the event table was hash-partitioned on match_id across two PostgreSQL servers through postgres_fdw (617 and 523 matches). A per-match query touched one shard; a query without the key touched both; full scans were about 10x slower than on one node (52 ms against about 500 ms), so sharding buys capacity, not speed. Moving to three shards would relocate 65% of matches. **Backup:** pg_dump took 3.5 s (18 MB) and the restore 4.3 s, verified by row counts and a checksum over 1.14M rows. **Cloud:** a deployment design only, not provisioned for this portfolio: two availability domains, a private subnet, a managed-versus-self-managed comparison, security controls and a runbook (docs/cloud-deployment.md). The mechanics it relies on (base backup, replica, promotion, restore) were measured above.

7. Individual technical reflection

My design choices were driven by the real workload: The Data XI is based on real project and it reads the same few things constantly (leaderboards, match pages), corrects a small number of scores, and holds one very large table. That is why PostgreSQL stays the source of truth, Redis sits in front for reads that tolerate staleness, and MongoDB is limited to read-mostly aggregates and raw json payloads.

The most useful lessons came from things going wrong. A cross-table integrity check found 398 matches that no CHECK constraint could see. An early benchmark showed that indexes didn't improve small queries, which taught me that an index is a cost as well as a benefit. A synchronous-replication demo hung because statement_timeout does not interrupt the commit wait. Grouping a leaderboard by name instead of id merged different players. Sharding made full scans slower, not faster. Each of these corrected an assumption I held before building this project.

If I had more time,I would add WAL archiving for point-in-time recovery, test automated failover, and measure the same experiments on a real two-machine setup where network latency is genuine.

8. AI use declaration

Tool: Claude (Anthropic). Used for: drafting the project and some sample files in SQL, Python, the data generator and some documentation. Verification: I ran and built everything step by step myself on my own macbook (Docker: PostgreSQL 16, Redis 7, MongoDB 7.0.45); the result files in the repository are my runs and the numbers in this report were checked against them, the numbers and results may vary depending on the machine.

9. References

MIVA Open University (2025) MIT 8103 Advanced Database Systems, Study Sessions 1 to 6. | MongoDB Inc. MongoDB Manual, https://www.mongodb.com/docs/manual/. | PostgreSQL Global Development Group. PostgreSQL 16 Documentation, https://www.postgresql.org/docs/16/. | Redis Ltd. Redis documentation, https://redis.io/docs/.